

7.性能优化实战：合成层的妙用

核心知识点

- `will-change` 的使用
- 合成层的利与弊

一、如何创建合成层？

浏览器在渲染过程中，可能会自动为某些元素创建合成层，这被称为**隐式合成**。但作为开发者，我们有时需要更精确地控制，这就是**显式合成**。

1. 隐式合成 (Implicit Compositing)

在上一节我们提到，当一个元素满足特定条件时，浏览器会“智能地”将其提升到一个独立的合成层。常见触发条件包括：

- 3D变换： `transform: translate3d(0, 0, 0);`， `transform: rotate3d(...)` 等。
- `position: fixed;`
- 拥有 `opacity` 或 `transform` 动画的元素。
- `<video>`，`<canvas>`，`<iframe>` 元素。
- 使用了 `filter` 属性的元素。

在过去，开发者会使用一些“hack”手段来强制创建合成层，最著名的就是 `transform: translateZ(0);` 或 `transform: translate3d(0, 0, 0);`。虽然这能起作用，但其语义不清晰，且不够规范。

2. 显式合成： `will-change`

为了解决上述 hack 的问题，CSS 引入了 `will-change` 属性。这是一个专门为性能优化设计的“提示性”属性。

`will-change` 的作用是提前告知浏览器：“嘿，这个元素的某个属性即将发生变化，请你提前做好准备和优化！”

如何使用？

它的值是你期望改变的 CSS 属性名。

代码块

```
1  /* 告诉浏览器，这个元素的 transform 属性即将改变 */
2  .animating-element {
3      will-change: transform;
4  }
5
6  /* 也可以指定多个属性 */
7  .another-element {
8      will-change: transform, opacity;
9  }
```

当浏览器看到 `will-change: transform;` 时，它通常会提前将该元素提升到一个独立的合成层，这样当 `transform` 真的发生变化时，就可以直接进入高效的“合成”阶段，跳过布局和绘制。

二、合成层的利与弊

使用合成层就像一把双刃剑，用得好能大幅提升性能，用得不好则会适得其反。

合成层的好处

1. **极致流畅的动画:** 这是最大的好处。将动画元素提升到合成层后，其 `transform` 和 `opacity` 的变化由 GPU 直接处理，可以轻松实现 60fps 的丝滑动画，避免了主线程的计算压力，从而避免了页面卡顿。

2. **隔离影响范围:** 当一个元素在自己的合成层上运动时，它不会引发整个页面的重绘（Repaint）或回流（Reflow）。这就像它在一个独立的画板上作画，不会影响到背景画板，极大地减少了不必要的渲染开销。

合成层的“弊”

1. **内存消耗剧增:** 每创建一个新的合成层，就意味着需要分配新的内存（RAM）和显存（VRAM）来存储这个图层的位图信息。如果页面上存在大量合成层，内存占用会急剧上升，尤其是在移动端和低端设备上，可能会导致页面响应变慢甚至崩溃。
2. **初始化开销:** 将一个元素提升到合成层并非“免费”的操作。浏览器需要为其分配内存、生成纹理、并将其上传到 GPU。这个过程本身需要时间和计算资源，有时可能会导致页面初次渲染时出现微小的延迟或闪烁。
3. **潜在的渲染问题:** 在某些情况下，强制提升图层可能会导致一些视觉上的小问题，例如字体渲染可能会变得有些模糊，因为文本被当作位图纹理进行了处理。

三、最佳实践与总结

1. **不要过早优化，更不要滥用:** 不要给页面上所有可能动的元素都加上 `will-change`。只有当一个复杂的动画确实出现了性能问题时，才考虑使用它进行优化。
2. **用完就“扔”:** `will-change` 的最佳实践是“按需使用”。通过 JavaScript 在动画即将开始时添加该属性，在动画结束后移除它。

代码块

```
1  const element = document.querySelector('.my-element');
2
3  // 动画开始前
4  element.addEventListener('mouseenter', () => {
5    element.style.willChange = 'transform';
6  });
7
8  // 动画结束后
9  element.addEventListener('animationend', () => {
10    element.style.willChange = 'auto'; // 'auto' 是默认值
11  });
```

3. **借助开发者工具：**使用 Chrome 开发者工具的 `Layers` (图层) 面板。这是一个强大的工具，你可以实时查看页面上的合成层情况，诊断哪些元素被提升了，以及是否存在不必要的层，从而帮助你做出正确的优化决策。

结论：合成层是实现高性能前端动画的关键技术。通过理解其工作原理，并明智、审慎地使用 `will-change`，你可以在保证性能的同时，避免其带来的副作用，创造出既华丽又流畅的用户体验。